Project / EGT216

Admin No. *233523A*

NANYANG POLYTECHNIC

2024/2025 SEMESTER 2 – PROJECT

Course : Diploma in AI and Data Engineering

Module : EGT216 - Natural Language Processing

Project / EGT216

Admin No. _____

1. Process Flow Diagram – Pre-processing

- Correct Encoding
- Unnecessary Character Removal
- Word Tokenization
- Lemmatization
- Stop Word Removal
- Vectorization

Admin No. _____

2. Pre-processing

a. Technique a

Code Snippet

```
initial_size = df.shape[0]
def track_data_size(initial_size, operation, prev_drop=0):
  num_dropped = initial_size - df.shape[0] + prev_drop
  print(f"Dropped {num_dropped} records during {operation}.")
  print(f"Percentage dropped: {(num_dropped/initial_size)*100:.2f}%")
  return num_dropped

df.dropna(inplace=True)
null_count = track_data_size(initial_size, "NULL drop")
assert df.isnull().sum().sum() == 0
```

```
Dropped 0 records during NULL drop.
Percentage dropped: 0.00%
```

```
df.drop_duplicates(inplace=True)
dul_count = track_data_size(initial_size, 'DROP DUPLICATES', null_count)
assert df.duplicated().sum() == 0

# drop num_words
df.drop('num_words', axis=1, inplace=True)
```

```
Dropped 744 records during DROP DUPLICATES.
Percentage dropped: 77.26%
```

Justification

Although dropping null and duplicates is not natively an NLP pre-processing technique, it is necessary for any data process, whether it's text or numbers. The dataset had no null values but had 77% duplicates.

the duplicates are quite problematic as they are 'actual' duplicates
they will be deleted, assuming these duplicates could be a result of
1. job applicants spamming the company's job application portal by using bots to submit the their same resume multiple times
2. another AI enginner colleague accidently creating duplicates due to an error in code
or, maybe coincidentally, some people with same qualifications are applying for the same role

not deleting these may affect the project by
1. introducing bias in later NLP models
2. wasting unnecessary computing resources
3. no point in analyzing the same thing as we are not interested in how common a resume is for a certain job role

b. Technique b

Code Snippet

Admin No. _____

```python
def clean_resume(text):
    # Step 1: Replace special symbols with their meanings
    text = re.sub(r'&|<|>|/|\\', lambda m: {'&': ' and ',
                                            '<': ' less than ',
                                            '>': ' greater than ',
                                            '/': ' or ',
                                            '\\': ' or '}[m.group()], text)

    # Step 2: Remove non-ASCII characters
    text = re.sub(r'[^\x00-\x7F]+', ' ', text)

    # Step 3: Remove control characters (newlines, tabs, etc.)
    text = re.sub(r'[\r\n\t]', ' ', text)

    # Step 4: Remove random symbols (excluding word characters and spaces)
    text = re.sub(r'[-±:`~!^*()_=]', ' ', text)

    # Step 5: Deal with special 'words' like C++ C#
    valid_cpp_variations = [
        "C++", "c++", "C ++", "c ++", "C++-", "c++-", "C ++-", "c ++-",
        "C++,", "c++,", "C ++,", "c ++,", "C++.", "c++.", "C ++.", "c ++.",
        "C#", "c#", "C #", "c #", "C#-", "c#-", "C #-", "c #-",
        "C#,", "c#,", "C #,", "c #,", "C#.", "c#.", "C #.", "c #."
    ]

    # Temporarily replace valid "C++" variations with placeholders
    for variant in valid_cpp_variations:
        text = text.replace(variant, "CXXPLUSXX")

    # Normalize "C#" variations before further processing
    text = re.sub(r'\b[Cc]\s*#\b', 'C#', text)  # Normalize variations like "C #", "c#"

    # Temporarily replace "C#" to protect it
    text = text.replace("C#", "CXXHASHXX")

    # Step 6: Remove standalone "++" and "#" (not part of protected "C++" or "C#")
    text = re.sub(r'(?<!C)(?<!c)\+\+', '', text)
    text = re.sub(r'(?<!C)(?<!c)#', '', text)

    # Step 7: Remove all other punctuation except words and spaces
    text = re.sub(r'[^\w\s]', ' ', text)

    # Step 8: Restore "C++" and "C#" from placeholders
    text = text.replace("CXXPLUSXX", "C++")
    text = text.replace("CXXHASHXX", "C#")

    return text

# Apply the function to the dataframe
df['resume'] = df['resume'].apply(clean_resume)
```

Justification
Some special characters that have meaning like ‘and’ for ‘&’ are replaced accordingly.
All non-ASCII, control and random characters are removed.
Some special words like C++ and C# are maintained as they are part of keywords in our case.

c. Technique c

Code Snippet

```
# Function to extract OOV words from text
def find_oov_words(text):
    tokens = re.findall(r'\b[a-zA-Z0-9]+\b', text)  # Extract words efficiently
    lemmatized_tokens = [lemmatizer.lemmatize(word.lower()) for word in tokens]
    return [word.lower() for word in tokens if word.lower() not in nltk_vocab]

# Apply function to all resumes and get a single list of OOV words
all_oov_words = set()  # Use a set to ensure uniqueness
df["resume"].apply(lambda text: all_oov_words.update(find_oov_words(text)))

# Convert set to sorted list
unique_oov_words = sorted(all_oov_words)

# Inspect result
columns = 6
oov_count = len(unique_oov_words)
remainder = oov_count % columns
if remainder != 0:
    for i in range(columns - remainder):
      unique_oov_words += ['']
oov_array = np.array(unique_oov_words)
oov_array = np.reshape(oov_array, (-1, columns))  # Reshape into multiple columns

print(pd.DataFrame(oov_array).to_string(index=False, header=False))
```

```
       transitions          translating          transmitters         transporters
             trays                  trb                 trees            trenching
         treplicate                trexo                trials            tribunals
           trimurti            trippereri                 trips              trombay
```

```
# quite subkective; so only those that are super obiviously random will be deleted.
# a lot of indian words and acronyms are detected.
imp_words = ["0", "000", "aofpt5052c", "kw", "yyyy", "mm", "dd", "F5qJ2rH7B0"]

for word in imp_words:
    try:
        unique_oov_words.remove(word)
    except ValueError:
        pass

# remove imp_words
df['resume'] = df['resume'].apply(lambda x: ' '.join([word for word in x.split() if word not in
imp_words]))

# duplicates might appear again after these preprocessing techniques
df.duplicated().sum()
```

```
35
```

```
# as expected; drop them
df.drop_duplicates(inplace=True)
assert df.duplicated().sum() == 0
assert df.isnull().sum().sum() == 0
```

Justification
There is another interesting issue -- random characters like 'f5qj2rh7b0' and deleting these 'non-meaningful' words would be quite tricky to deal with unless we have a comprehensive vocabulary corpus that includes modern words and acronyms like blockchain, erp, sap....
Another comprehensive and modern vocabulary corpus could have been used, but since we are limited to nltk, OOV words not in nltk vocab are manually extracted and relevant non-meaningful noise words are removed.
Duplicates that appear after this ‘fake word deletion’ are deleted as well.

d. Technique d

Code Snippet

Admin No. _____

```
"""Tokenization"""
# character tokenization is way too primitive and uneffective as it's difficult to inference any
meaning from characters alone
# either word or subword tokenization
# word tokenization is used for its simple interpretability

df['tokens'] = df['resume'].apply(lambda x: nltk.word_tokenize(x))

df.head()
```

	category	resume	tokens
0	Hadoop	Technical Skill Set Programming Languages Apac...	[Technical, Skill, Set, Programming, Languages...
1	Mechanical Engineer	SKILLS Knowledge of software or computer Auto ...	[SKILLS, Knowledge, of, software, or, computer...
2	Testing	Skill Set OS Windows XP or 7 or 8 or 8 1 or 10...	[Skill, Set, OS, Windows, XP, or, 7, or, 8, or...
3	Electrical Engineering	Education Details January 2012 to January 2013...	[Education, Details, January, 2012, to, Januar...
4	Business Analyst	Key Skills Requirement Gathering Requirement A...	[Key, Skills, Requirement, Gathering, Requirem...

Justification

Simple word tokenization is used over other techniques due to its interpretability and for compatibility with BrightSpace's Bert code for later use.

e. Technique e

Code Snippet

```
"""Lemmatization"""

# Extract uppercase words from resume (potential acronyms)
def extract_acronyms(text):
    return set(re.findall(r'\b[A-Z]{2,}\b', text))  # Finds words with ALL CAPS (2+ letters)

# Get WordNet POS tag
def get_wordnet_pos(word):
    tag = nltk.pos_tag([word])[0][1][0].upper()
    tag_dict = {"J": wordnet.ADJ, "N": wordnet.NOUN, "V": wordnet.VERB, "R": wordnet.ADV}
    return tag_dict.get(tag, None)  # Only return POS tag if found

# Lemmatize words while preserving acronyms
def lemmatize_words(tokens, acronyms):
    lemmatized_tokens = []
    for word in tokens:
        if word.upper() in acronyms:  # If word is in extracted acronyms, keep it
            lemmatized_tokens.append(word.upper())  # Preserve original uppercase
        else:
            pos = get_wordnet_pos(word)  # Get POS tag
            if pos:
                lemmatized_tokens.append(lemmatizer.lemmatize(word.lower(), pos))  # Lemmatize if
                POS is valid
            else:
                lemmatized_tokens.append(word.lower())  # Return original word if POS is unknown
    return lemmatized_tokens

# Apply the functions
df["Acronyms"] = df["resume"].apply(lambda x: extract_acronyms(str(x)))  # Extract acronyms per
resume
df["tokens_cleaned"] = df.apply(lambda row: lemmatize_words(row["tokens"], row["Acronyms"]), axis=1)

# Drop acronyms
df.drop(["Acronyms"], axis=1, inplace=True)
```

Justification

The usual lemmatisation is not performing well; it removed 's' from 'os' which we don't want. Thus, lemmatization is done only if POS tag is known and original word is retained otherwise. Some important keywords like OS, XP, ERP are assumed to be in upper-case in formal resume and this those acronyms in uppercase are trained as well.

f. Technique f

Code Snippet

```
# stopwords removal
def remove_stopwords_from_list(token_list):
    return [token for token in token_list if token not in stop_words]

df['tokens_cleaned'] = df['tokens_cleaned'].apply(remove_stopwords_from_list)

df.head()
```

	category	resume	tokens	tokens_cleaned
0	Hadoop	Technical Skill Set Programming Languages Apac...	[Technical, Skill, Set, Programming, Languages...	[technical, skill, set, program, language, APA...
1	Mechanical Engineer	SKILLS Knowledge of software or computer Auto ...	[SKILLS, Knowledge, of, software, or, computer...	[SKILLS, knowledge, software, computer, AUTO, ...
2	Testing	Skill Set OS Windows XP or 7 or 8 or 8 1 or 10...	[Skill, Set, OS, Windows, XP, or, 7, or, 8, or...	[skill, set, OS, WINDOWS, XP, 7, 8, 8, 1, 10, ...
3	Electrical Engineering	Education Details January 2012 to January 2013...	[Education, Details, January, 2012, to, Januar...	[education, detail, january, 2012, january, 20...
4	Business Analyst	Key Skills Requirement Gathering Requirement A...	[Key, Skills, Requirement, Gathering, Requirem...	[key, skill, REQUIREMENT, GATHERING, REQUIREME...

Justification

Stop words are removed to shrink down word count for later vectorization and model training and to make the model focus on keywords rather than common non-value-added words. (not necessary for BERT though, but still)

3. Vectorization

a. Technique a

Code Snippet

```
# One-hot and BOW are way too sparse
# Vectorizations to test out: TF-IDF, PPMI, Word2Vec, BERT

"""TF-IDF"""
# Calculate TF
def calculate_tf(value):
  tf = np.log10(value + 1)
  return tf

# Calculate IDF
def calculate_idf(total_documents, value):
  idf = np.log10(total_documents / value)
  return idf

# focusing only on keywords similarity; not years of experience nor result metrics
def concatanate_words_only(word_list):
    cleaned_words = []
    for word in word_list:
        cleaned_word = re.sub(r'\d', '', word) # remove digits
        if cleaned_word:
            cleaned_words.append(cleaned_word) # drop null [2024 --> '' --> removed]
    return " ".join(cleaned_words)

def tfidf_similarity(documents):
  # Create TDM
  X = vectorizer.fit_transform(documents)
  tdm = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())

  # Create IDF array
  total_documents = len(documents)
  total_documents_per_word = np.sum(tdm > 0, axis=0)
  idf_array = total_documents_per_word.apply(lambda value:
  calculate_idf(total_documents, value))
```

```
  # Create TF matrix
  tf_matrix = tdm.map(calculate_tf)

  # Create TF-IDF matrix
  tfidf_matrix = tf_matrix * idf_array.to_numpy()

  # calculate cosine similarity
  cosine_matrix = cosine_similarity(tfidf_matrix)

  # take only the last row to compare against my own resume
  return tfidf_matrix, cosine_matrix[-1]

vectorizer = CountVectorizer()
tfidf_start = time.time()
documents_words = df['tokens_cleaned'].apply(concatanate_words_only)
tfidf, cosine = tfidf_similarity(documents_words)
tfidf_duration = time.time() - tfidf_start
print(f"Time taken for TF-IDF: {tfidf_duration:.2f} seconds.")
print('\n')

# add the cosine similarity as a new column
df['tfidf_score'] = cosine
df.reset_index(drop=True, inplace=True)
df.sort_values(by=['tfidf_score'], ascending=False).head()
```

```
Time taken for TF-IDF: 3.66 seconds.
```

	category	tokens_cleaned	tfidf_score
183	Min Phyo Thura	[phyo, thura, machine, learn, engineer, 65, 80...	1.000000
129	Data Science	[personal, skill, ability, quickly, grasp, tec...	0.108781
88	Data Science	[education, detail, b, tech, rayat, AND, bahra...	0.098442
65	Data Science	[expertise, data, quantitative, analysis, deci...	0.093285

Justification

TD-IDF vectorization is manually done using TF and IDF separately instead of directly using TFIDFVectorizer from sklearn. TFIDF vectorization lowers the importance of common words while increasing the importance of unique words and is thus commonly used and effective in document classification like resume classification.

b. Technique a

Code Snippet

Admin No.

```
ppmi_start = time.time()

# Extract all tokens from df['tokens_cleaned']
all_tokens = df['tokens_cleaned'].explode().tolist()

# Create vocabulary
vocabulary = list(set(all_tokens))
vocab_index = {word: i for i, word in enumerate(vocabulary)}  # Word to index mapping

# Define context window size (e.g., ±5 words)
window_size = 5

# Initialize co-occurrence matrix
co_occurrence_matrix = np.zeros((len(vocabulary), len(vocabulary)), dtype=np.float64)

# Compute co-occurrence counts efficiently
for tokens in df['tokens_cleaned']:
    for i, token in enumerate(tokens):
        token_idx = vocab_index[token]
        left_window = max(i - window_size, 0)
        right_window = min(i + window_size + 1, len(tokens))

        for j in range(left_window, right_window):
            if i != j:  # Avoid self-co-occurrence
                co_token_idx = vocab_index[tokens[j]]
                co_occurrence_matrix[token_idx, co_token_idx] += 1

# Compute probabilities
total_co_occurrences = np.sum(co_occurrence_matrix)
word_freqs = np.sum(co_occurrence_matrix, axis=1)  # Sum over rows for p(word)
word_probs = word_freqs / total_co_occurrences  # P(w)
```

```
# Compute probabilities
total_co_occurrences = np.sum(co_occurrence_matrix)
word_freqs = np.sum(co_occurrence_matrix, axis=1)  # Sum over rows for p(word)
word_probs = word_freqs / total_co_occurrences  # P(w)

# Compute PPMI matrix
ppmi_matrix = np.zeros_like(co_occurrence_matrix)

for i in range(len(vocabulary)):
    for j in range(len(vocabulary)):
        if co_occurrence_matrix[i, j] > 0:
            p_ij = co_occurrence_matrix[i, j] / total_co_occurrences  # P(word1, word2)
            p_i = word_probs[i]
            p_j = word_probs[j]
            pmi = np.log2(p_ij / (p_i * p_j)) if p_ij > 0 else 0
            ppmi_matrix[i, j] = max(pmi, 0)  # Apply Positive PMI

ppmi_duration_1 = time.time() - ppmi_start

# Convert to Pandas DataFrame for visualization
ppmi_df = pd.DataFrame(ppmi_matrix, index=vocabulary, columns=vocabulary)
ppmi_df.head()
```

	tikona	jsp	teleconference	custody	journalist	nature	sincerity	juniper	MSCIT	upload	...	megger	FIELD	logix5000	weblogic	pivot	safety	firre	ALAMURI
tikona	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
jsp	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
teleconference	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

```
ppmi_start = time.time()

# Get the vocabulary index from the existing PPMI DataFrame
vocab = ppmi_df.index.to_list()
vocab_index = {word: i for i, word in enumerate(vocab)}

# Function to get a resume-level PPMI vector
def get_resume_vector(tokens, ppmi_df):
    vector = np.zeros(len(vocab))
    for token in tokens:
        if token in vocab_index:  # Only include known words
            vector += ppmi_df.iloc[vocab_index[token]].values
    return vector

# Compute PPMI vectors for all resumes
df["ppmi_vector"] = df["tokens_cleaned"].apply(lambda tokens: get_resume_vector(tokens, ppmi_df))

# Get the vector for the last resume in the dataset
last_resume_vector = df.iloc[-1]["ppmi_vector"].reshape(1, -1)  # Ensure it's 2D for cosine
similarity

# Compute cosine similarity between last resume and all others
cosine_similarities = cosine_similarity(last_resume_vector, np.vstack(df["ppmi_vector"]))[0]

ppmi_duration = time.time() - ppmi_start + ppmi_duration_1
print(f"Time taken for PPMI: {ppmi_duration:.2f} seconds.")
print('\n')

# Add similarity scores to the DataFrame
df["ppmi_score"] = cosine_similarities

# Drop ppmi_vector
df.drop(["ppmi_vector"], axis=1, inplace=True)

# Sort by similarity score (descending)
df.sort_values(by="ppmi_score", ascending=False).head()
```

```
Time taken for PPMI: 26.03 seconds.
```

	category	tokens_cleaned	tfidf_score	ppmi_score
183	Min Phyo Thura	[phyo, thura, machine, learn, engineer, 65, 80...	1.000000	1.000000
65	Data Science	[expertise, data, quantitative, analysis, deci...	0.093285	0.728777
129	Data Science	[personal, skill, ability, quickly, grasp, tec...	0.108781	0.721465
88	Data Science	[education, detail, b, tech, rayat, AND, bahra...	0.098442	0.688043
13	Data Science	[skill, r, python, SAP, HANA, tableau, SAP, HA...	0.083900	0.675710

Justification
PPMI is also an effective vectorization for document classification tasks as it provides the probability of two words appearing together than expected by chance, thereby providing info for key words that appear together in a resume.

c. Technique a

Code Snippet

```python
# Function to get sentence vector by averaging word vectors
def get_sentence_vector(tokens, model):
    vectors = [model[word] for word in tokens if word in model]
    return np.mean(vectors, axis=0) if vectors else np.zeros(model.vector_size)

# Apply function to get vector representation
word2vec_start = time.time()
df["word2vec"] = df["tokens_cleaned"].apply(lambda tokens: get_sentence_vector(tokens,
word2vec_model))
word2vec_duration_1 = time.time() - word2vec_start
df.head()["word2vec"]
```

	word2vec
0	[0.31880167, 0.036123544, 0.21636704, 0.159727...
1	[-0.07394834, 0.056731537, 0.09178243, 0.11679...
2	[0.19753347, 0.21969093, 0.29083788, 0.3160259...
3	[0.13110745, -0.04714064, 0.3042061, 0.1585521...
4	[0.28354773, 0.0006305353, 0.08927156, 0.18197...

```
word2vec_start = time.time()

# Extract vectors as a numpy array
vectors = np.vstack(df["word2vec"].values)

# Get the last row's vector
my_cv = vectors[-1].reshape(1, -1)  # Reshape for sklearn compatibility

# Compute cosine similarity with all other vectors
cosine_similarities = cosine_similarity(my_cv, vectors).flatten()

word2vec_duration = time.time() - word2vec_start + word2vec_duration_1
print(f"Time taken for Word2Vec: {word2vec_duration:.2f} seconds.")
print('\n')

# Add cosine similarity scores to the dataframe
df["word2vec_score"] = cosine_similarities

# Drop word2vec column
df.drop(["word2vec"], axis=1, inplace=True)

# Sort by similarity score (descending)
df.sort_values(by="word2vec_score", ascending=False).head()
```

```
Time taken for Word2Vec: 0.48 seconds.
```

	category	tokens_cleaned	tfidf_score	ppmi_score	word2vec_score
183	Min Phyo Thura	[phyo, thura, machine, learn, engineer, 65, 80...	1.000000	1.000000	1.000000
65	Data Science	[expertise, data, quantitative, analysis, deci...	0.093285	0.728777	0.982538
129	Data Science	[personal, skill, ability, quickly, grasp, tec...	0.108781	0.721465	0.981804
53	Automation Testing	[technical, skill, summary, complete, CORPORAT...	0.035541	0.552716	0.979179
71	Database	[TECHNICAL, SKILLS, SQL, ORACLE, v10, v11, v12...	0.076718	0.636069	0.975804

Justification

Word2Vec captures semantic relationships by mapping words into dense vectors, thereby improving resume classification based on contextual similarities.

d. Technique a

Code Snippet

BERT

```
# Function to get BERT embeddings
def get_bert_embedding(tokens, model, tokenizer):
    text = " ".join(tokens)  # Convert list of tokens into a string
    inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True, max_length=128)

    with torch.no_grad():  # No gradients needed
        outputs = model(**inputs)

    # Extract the embeddings of [CLS] token (sentence-level representation)
    cls_embedding = outputs.last_hidden_state[:, 0, :].squeeze().numpy()

    return cls_embedding

bert_start = time.time()

# Apply function to dataset
df["bert_vector"] = df["tokens_cleaned"].apply(lambda tokens: get_bert_embedding(tokens,
bert_model, tokenizer))

# Extract vectors as a numpy array
vectors = np.vstack(df["bert_vector"].values)

# Get the last row's vector
my_cv = vectors[-1].reshape(1, -1)  # Reshape for sklearn compatibility

# Compute cosine similarity with all other vectors
cosine_similarities = cosine_similarity(my_cv, vectors).flatten()

bert_duration = time.time() - bert_start
print(f"Time taken for BERT: {bert_duration:.2f} seconds.")
print('\n')
```

```
bert_duration = time.time() - bert_start
print(f"Time taken for BERT: {bert_duration:.2f} seconds.")
print('\n')

# Add cosine similarity scores to the dataframe
df["bert_score"] = cosine_similarities

# Drop word2vec column
df.drop(["bert_vector"], axis=1, inplace=True)

# Sort by similarity score (descending)
df.sort_values(by="bert_score", ascending=False).head()
```

```
Time taken for BERT: 102.11 seconds.
```

	category	tokens_cleaned	tfidf_score	ppmi_score	word2vec_score	bert_score
183	Min Phyo Thura	[phyo, thura, machine, learn, engineer, 65, 80...	1.000000	1.000000	1.000000	1.000000
6	Blockchain	[SKILLS, bitcoin, ethereum, solidity, hyperled...	0.043059	0.473702	0.959035	0.922691
120	Blockchain	[SKILLS, bitcoin, ethereum, solidity, hyperled...	0.043002	0.474189	0.959035	0.922691

Justification

BERT is the most popular attention-based embeddings-encoder and excels in resume classification by understanding word context bidirectionally. Its deep encoding captures nuanced job-related terms, enhancing classification accuracy.

4. Resume Ranking

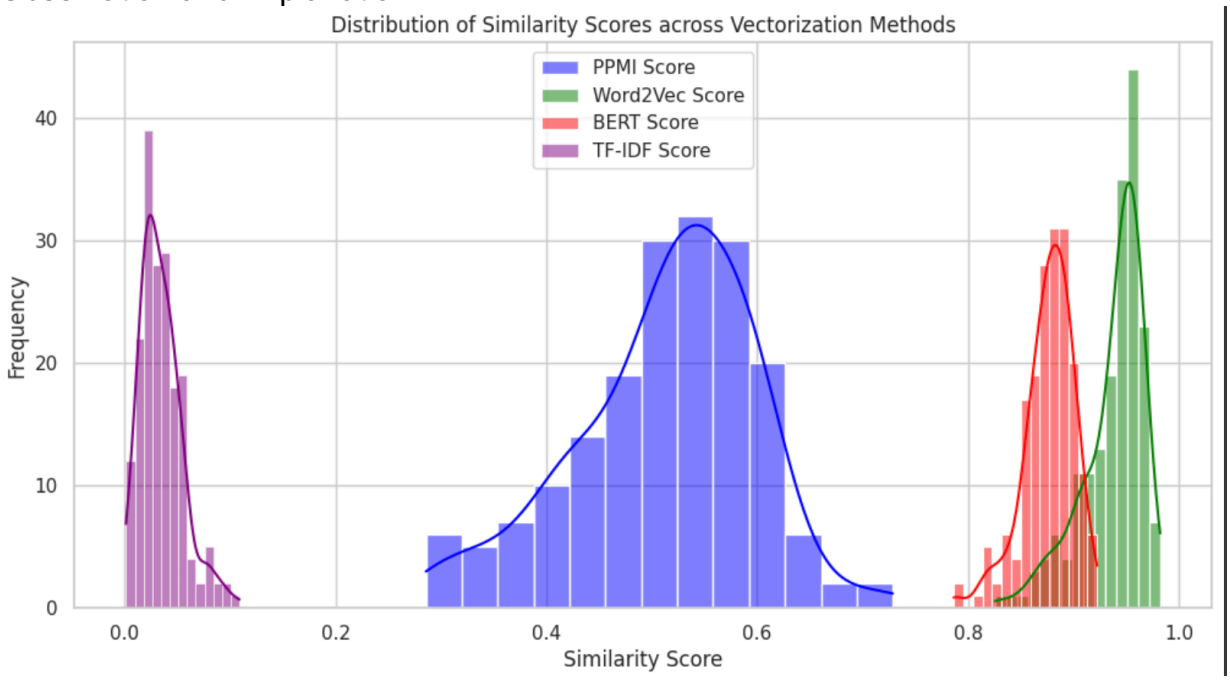
Code Snippet

The previous codes ranks the resumes using sort-by after each vectorization. So, graphs are provided here for better comparison and understanding through visuals.

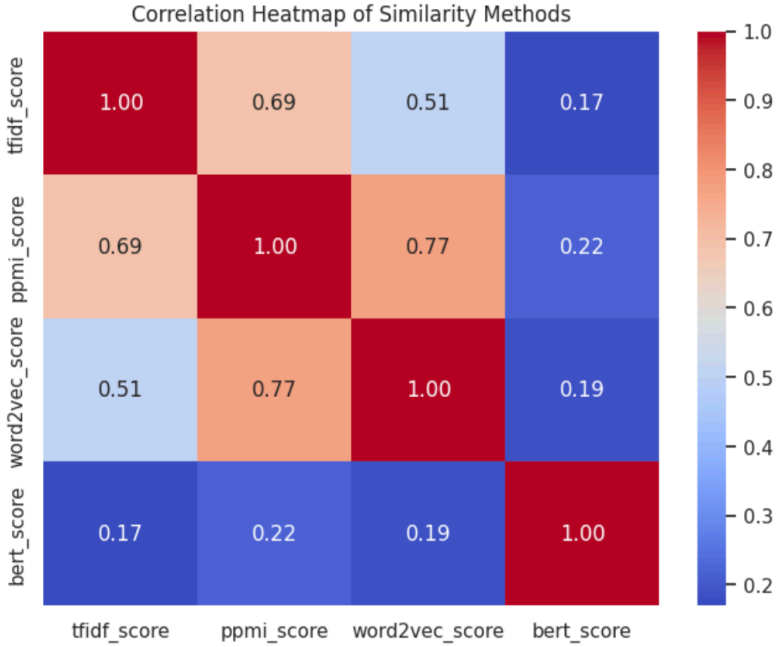
Project / EGT216

Admin No. _____

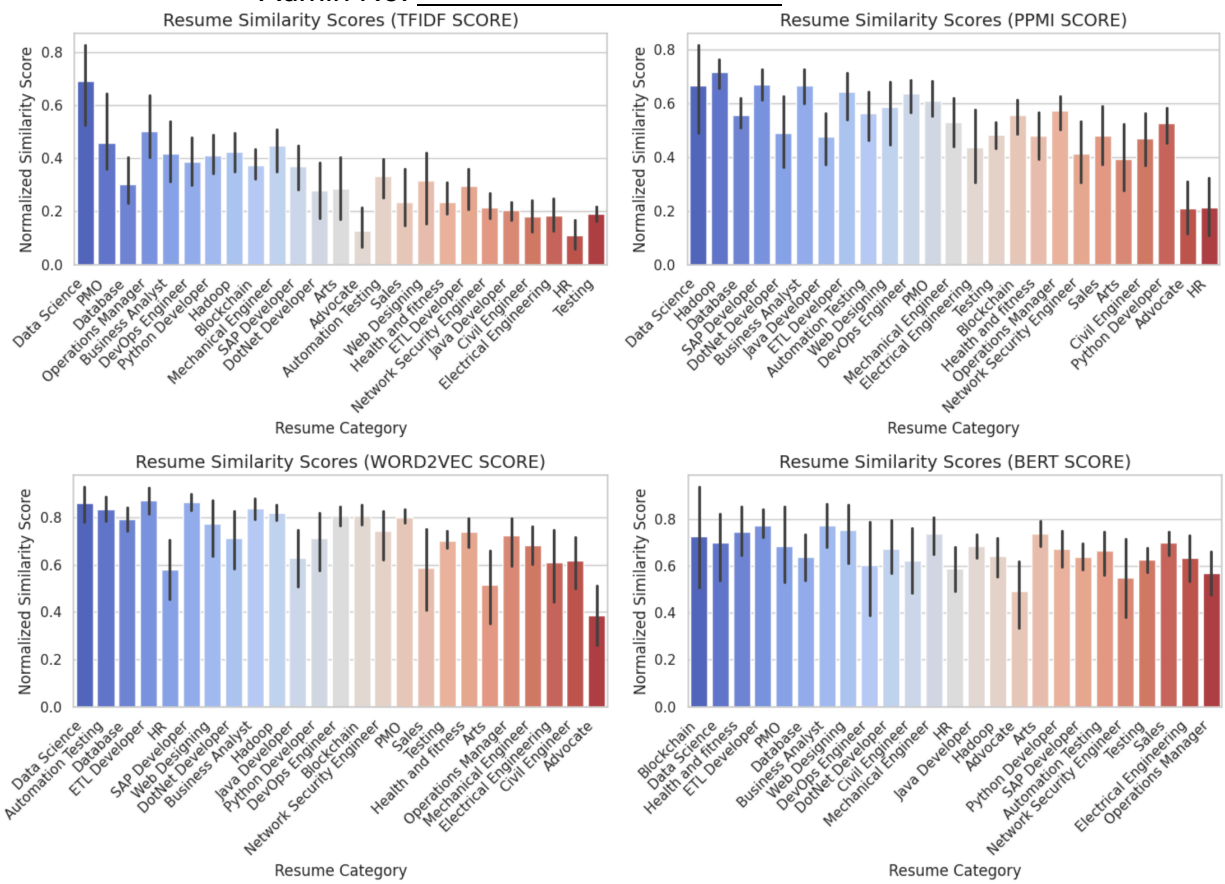
Observation and Explanation



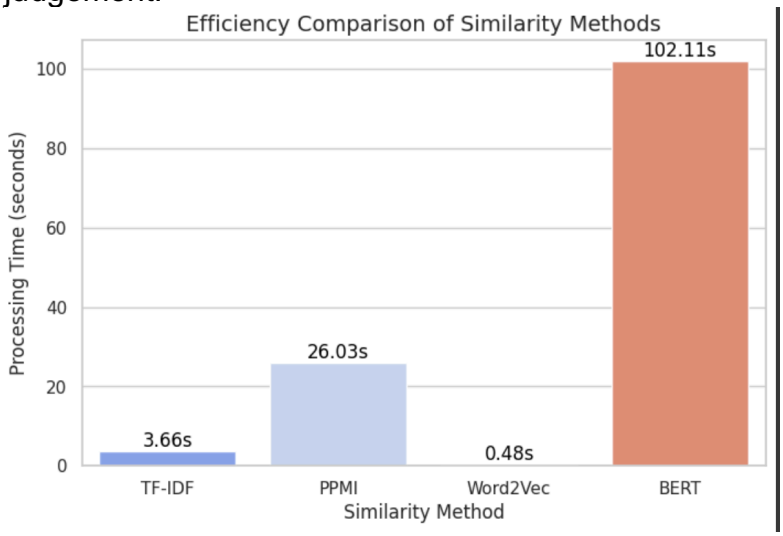
[From left to right] Tf-idf scores are quite low in the [0-1] range, meaning the algorithm is not confident in its similarity. PPMI scores are more well-distributed and seem to be performing well. Both Word2Vec and BERT are quite confident in their similarity, but their distribution is densely packed together, an indication that it would struggle to differentiate between similarly-scored resumes.



Checking the similarity scores correlations, PPMI and Word2Vec have the highest covariance, meaning if a resume in PPMI vectorization has a high similarity score, it might also have a high score in Word2Vec vectorization as well.



In these charts, left-most job roles have the highest similarity score to my cv (which is aimed to a Data Scientist & Data/Analysis related with leadership skills). From above, TFIDF provides the best similarity ranks as per human judgement.



This is the time taken to generate embeddings and calculate cosine similarity for each vectorization technique, and it can be seen that TF-IDF and Word2Vec are the most efficient ones.

Conclusion: Given its efficient computing and accurate search of similar job roles and cv, TD-IDF is chosen to be the best vectorization technique for this dataset and this usage.

5. BERT Fine-Tuning

Code Snippet

```python
# Define max words per row and overlap size
MAX_WORDS = 185 # 192 words for 256 tokens
OVERLAP = 20  # Overlap 20 words from the previous chunk

# Function to split resume text with overlap
def split_resume(row):
    words = row["resume"].split()  # Tokenize by whitespace
    if len(words) <= MAX_WORDS:
        return [row]  # Keep as is if under the limit

    # Create overlapping chunks
    chunks = [words[i:i + MAX_WORDS] for i in range(0, len(words), MAX_WORDS - OVERLAP)]

    # Create new rows for each chunk
    new_rows = [{"category": row["category"], "resume": " ".join(chunk)} for chunk in chunks]
    return new_rows

# Apply function to all rows and expand into a new DataFrame
df_split = pd.DataFrame([row for rows in df.apply(split_resume, axis=1) for row in rows])

# Reset index for clean formatting
df_split.reset_index(drop=True, inplace=True)
```

```python
df = pd.read_csv("/kaggle/input/nlp-clean/BERT_data_final.csv", index_col=0)
df.head()
```

	category	resume
0	Hadoop	technical skill set program language APACHE HA...
1	Hadoop	service description project description data w...
2	Hadoop	order increase performance minimize HADOOP res...
3	Hadoop	objective project speed data processing analys...
4	Testing	good logical analytical skill positive attitud...

+ Code + Markdown

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 444 entries, 0 to 443
Data columns (total 2 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   category  444 non-null    object
 1   resume    444 non-null    object
dtypes: object(2)
memory usage: 10.4+ KB
```

Project / EGT216

Admin No. _____

```
data_frame = pd.DataFrame()
data_frame['sentence'] = df['resume']
data_frame['label'] = df['category']
data_frame.head()
```

	sentence	label
0	technical skill set program language APACHE HA...	Hadoop
1	service description project description data w...	Hadoop
2	order increase performance minimize HADOOP res...	Hadoop
3	objective project speed data processing analys...	Hadoop
4	good logical analytical skill positive attitud...	Testing

```
# Prepare features and labels for training and validation
sentences = data_frame.sentence.values
sentences = ["[CLS] " + sentence + " [SEP]" for sentence in sentences]
labels = data_frame.label.values
```

+ Code + Markdown

```
# Generate attention masks for training and validation
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased', do_lower_case=True)
tokenized_sentences = [tokenizer.tokenize(sentence) for sentence in sentences]
input_id_sequences = [tokenizer.convert_tokens_to_ids(tokenized_sentence) for tokenized_sentence in tokenized_se
input_id_sequences = pad_sequences(input_id_sequences, maxlen=256, dtype="long", truncating="post", padding="pos
attention_masks = [[float(i > 0) for i in input_id_sequence] for input_id_sequence in input_id_sequences]
```

```
# Prepare datasets for training and validation
training_dataset, validation_dataset, training_labels, validation_labels, training_masks, validation_masks = tra
labels, attention_masks, random_state=2018, test_size=0.1)

# Initialize the encoder
label_encoder = LabelEncoder()

# Fit and transform labels into numeric values
training_labels = label_encoder.fit_transform(training_labels)
validation_labels = label_encoder.transform(validation_labels)

# Ensure labels and masks are in proper NumPy formats
training_labels = np.array(training_labels, dtype=np.int64)  # Ensure int format
validation_labels = np.array(validation_labels, dtype=np.int64)

training_masks = np.array(training_masks, dtype=np.int64)  # Ensure int format
validation_masks = np.array(validation_masks, dtype=np.int64)

# Ensure input IDs are in correct format
training_dataset = np.array(training_dataset, dtype=np.int64)
validation_dataset = np.array(validation_dataset, dtype=np.int64)

batch_size = 16

training_dataset = TensorDataset(torch.tensor(training_dataset).to(device), torch.tensor(training_masks).to(devi
training_sampler = RandomSampler(training_dataset)
training_dataloader = DataLoader(training_dataset, sampler=training_sampler, batch_size=batch_size)

validation_dataset = TensorDataset(torch.tensor(validation_dataset).to(device), torch.tensor(validation_masks).t
validation_sampler = SequentialSampler(validation_dataset)
validation_dataloader = DataLoader(validation_dataset, sampler=validation_sampler, batch_size=batch_size)

num_classes = len(np.unique(training_labels))
```

Project / EGT216

Admin No. _____

Raw bert-base-uncased from BrightSpace

```
# Configure model
configuration = BertConfig()
model = BertModel(configuration)
config = model.config

model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=num_classes)
model = nn.DataParallel(model)
model.to(device)

parameters = list(model.named_parameters())
no_decay = ['bias', 'LayerNorm.weight']
parameters = [
    {'params': [p for n, p in parameters if not any(nd in n for nd in no_decay)], 'weight_decay': 0.01},
    {'params': [p for n, p in parameters if any(nd in n for nd in no_decay)], 'weight_decay': 0.0}
]
optimizer = AdamW(parameters, lr=2e-5, correct_bias=False)
```

```
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncase
d and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

+ Code + Markdown

```
# Calculate accuracy
def accuracy(predicted_labels, labels):
    predicted_labels = np.argmax(predicted_labels.to('cpu').numpy(), axis=1).flatten()
    labels = labels.to('cpu').numpy().flatten()
    return np.sum(predicted_labels == labels) / len(labels)
```

```
import time

# Train model
total_epochs = 20
training_losses = []

start = time.time()
for epoch in trange(total_epochs, desc="Epoch"):
    model.train()
    training_loss = 0
    training_steps = 0
    training_correct = 0
    training_total = 0

    for step, batch in enumerate(training_dataloader):
        inputs = batch[0].to(device)
        attention_masks = batch[1].to(device)
        labels = batch[2].to(device)

        optimizer.zero_grad()
        outputs = model(inputs, attention_mask=attention_masks, labels=labels)
        loss = outputs.loss
        logits = outputs.logits

        loss.backward()
        optimizer.step()

        training_loss += loss.item()
        training_steps += 1

        training_losses.append(loss.item())
```

Project / EGT216

Admin No. _____

```
        preds = torch.argmax(logits, dim=1)
        training_correct += (preds == labels).sum().item()
        training_total += labels.size(0)

    average_training_loss = training_loss/training_steps
    training_accuracy = training_correct / training_total

    print("Epoch {}: Average Training Loss: {:.4f}".format(epoch+1, average_training_loss))
    print("Epoch {}: Training Accuracy: {:.4f}".format(epoch+1, training_accuracy))

    model.eval()
    validation_accuracy = 0
    validation_steps = 0

    for batch in validation_dataloader:
        inputs = batch[0].to(device)
        attention_masks = batch[1].to(device)
        labels = batch[2].to(device)

        with torch.no_grad():
            outputs = model(inputs, attention_mask=attention_masks, labels=labels)

        logits = outputs.logits
        temp_validation_accuracy = accuracy(logits, labels)
        validation_accuracy += temp_validation_accuracy
        validation_steps += 1

    average_validation_accuracy = validation_accuracy/validation_steps
    print("Epoch {}: Validation Accuracy: {:.4f}".format(epoch+1, average_validation_accuracy))

raw_duration = time.time() - start
```

```
Epoch:   0%|          | 0/20 [00:00<?, ?it/s]
Epoch 1: Average Training Loss: 2.9002
```

```
from torch.optim.lr_scheduler import CosineAnnealingLR

model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=num_classes,
    hidden_dropout_prob=0.3,  # Increased to 0.3 for better regularization
    attention_probs_dropout_prob=0.3  # Prevent overfitting in attention layers
)

model = nn.DataParallel(model)
model.to(device)

parameters = list(model.named_parameters())
no_decay = ['bias', 'LayerNorm.weight']
parameters = [
    {'params': [p for n, p in parameters if not any(nd in n for nd in no_decay)], 'weight_decay': 0.01},
    {'params': [p for n, p in parameters if any(nd in n for nd in no_decay)], 'weight_decay': 0.0}
]

optimizer = AdamW(parameters, lr=2e-5, correct_bias=False)

# Add a scheduler for learning rate decay
lr_scheduler = CosineAnnealingLR(optimizer, T_max=total_epochs, eta_min=1e-6)

# Freeze Lower BERT Layers
for param in model.module.bert.embeddings.parameters():
    param.requires_grad = False
for layer in model.module.bert.encoder.layer[:4]:  # Freeze first 8 layers
    for param in layer.parameters():
        param.requires_grad = False
```

```
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncase
d and are newly initialized: ['classifier.bias', 'classifier.weight']
```

Project / EGT216

Admin No. _____

```python
import torch.nn.utils as nn_utils

# Train model
total_epochs = 100 # Early-stopping will stop the training if necessary
training_losses = []

# Early-stopping variables
best_val_acc = 0
patience = 7  # Stop training if no improvement for 7 consecutive epochs
counter = 0

# Path to save the best model
best_model_path = "/kaggle/working/best_bert_model.pth"

start = time.time()
for epoch in trange(total_epochs, desc="Epoch"):
    model.train()
    training_loss = 0
    training_steps = 0
    training_correct = 0
    training_total = 0

    if epoch == 10:
        for param in model.module.bert.parameters():
            param.requires_grad = True
        print("Unfreezing all BERT layers")

    for step, batch in enumerate(training_dataloader):
        inputs = batch[0].to(device)
        attention_masks = batch[1].to(device)
        labels = batch[2].to(device)

        optimizer.zero_grad()
        outputs = model(inputs, attention_mask=attention_masks, labels=labels)
```

Admin No. _____

```
        optimizer.zero_grad()
        outputs = model(inputs, attention_mask=attention_masks, labels=labels)
        loss = outputs.loss
        logits = outputs.logits

        loss.backward()

        # Apply Gradient Clipping (New)
        nn_utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        optimizer.step()

        # Update learning rate scheduler
        lr_scheduler.step()

        training_loss += loss.item()
        training_steps += 1

        training_losses.append(loss.item())

        preds = torch.argmax(logits, dim=1)
        training_correct += (preds == labels).sum().item()
        training_total += labels.size(0)

    average_training_loss = training_loss/training_steps
    training_accuracy = training_correct / training_total

    print("Epoch {}: Average Training Loss: {:.4f}".format(epoch+1, average_training_loss))
    print("Epoch {}: Training Accuracy: {:.4f}".format(epoch+1, training_accuracy))

    model.eval()
    validation_accuracy = 0
    validation_steps = 0

    for batch in validation_dataloader:
```

Admin No. _____

```
        inputs = batch[0].to(device)
        attention_masks = batch[1].to(device)
        labels = batch[2].to(device)

        with torch.no_grad():
            outputs = model(inputs, attention_mask=attention_masks, labels=labels)

        logits = outputs.logits
        temp_validation_accuracy = accuracy(logits, labels)
        validation_accuracy += temp_validation_accuracy
        validation_steps += 1

    average_validation_accuracy = validation_accuracy/validation_steps
    print("Epoch {}: Validation Accuracy: {:.4f}".format(epoch+1, average_validation_accuracy))

    # Save best model
    if average_validation_accuracy > best_val_acc:
        best_val_acc = average_validation_accuracy
        counter = 0  # Reset patience counter

        # Save the model
        torch.save(model.state_dict(), best_model_path)
        print(f"Best model saved at epoch {epoch+1} with validation accuracy {best_val_acc:.4f}")

    else:
        counter += 1  # Increment counter if no improvement
        print(f"No improvement, patience count: {counter}/{patience}")

    # Early Stopping Check
    if counter >= patience:
        print("Early stopping triggered. Training stopped.")
        break  # Stop training if no improvement for n consecutive epochs
duration = time.time() - start
```

Admin No. _____

```python
# Disable gradient calculations for evaluation
with torch.no_grad():
    for batch in validation_dataloader:
        inputs = batch[0].to(device)
        attention_masks = batch[1].to(device)
        labels = batch[2].to(device)

        # Forward pass
        outputs = model(inputs, attention_mask=attention_masks, labels=labels)
        loss = outputs.loss
        logits = outputs.logits

        # Calculate loss
        total_loss += loss.item()

        # Get predictions
        preds = torch.argmax(logits, dim=1)

        # Update accuracy calculations
        correct_predictions += (preds == labels).sum().item()
        total_samples += labels.size(0)

# Compute average loss and accuracy
average_loss = total_loss / len(prediction_dataloader)
accuracy = correct_predictions / total_samples

print(f"Validatin Dataset Accuracy: {accuracy:.4f}")
print(f"Validation Dataset Loss: {average_loss:.4f}")
```

```
Validatin Dataset Accuracy: 0.9333
Validation Dataset Loss: 0.0184
```

```
Overall Accuracy: 0.8170
Overall Loss: 0.8932
Overall MCC: 0.8153
```

+ Code + Markdown

Observation and Explanation

BERT model has a max token length of 512 tokens while the resumes in dataset are quite long with most over 1000 words. Thus, the resumes are chunked into at most 185 words and a bert-base-uncased model is trained using that data with 256 max token length from BrightSpace code. The model works and has 'acceptable' validation accuracy but is obviously overfitted. Thus, the BERT model is fine-tuned with
hidden_dropout_prob and attention_probs_dropout_prob for better regularization and reducing overfitting,
freezing the first 4 out of 12 layers of BERT to retain original generalization,
early stopping and saving best model to prevent overfitting.
The model achieved 99% train accuracy and 93% val accuracy with slight overfitting. The model was tested on Lim Jin Bin's dataset (with his permission)

and achieved an average MCC score of 81.5, indicating a strong performance.

6. Optimization

a. Technique a

Code

```
# Load the data as earlier but with 128 max token length
def load_data(path, token_length):
    df = pd.read_csv(path, index_col=0)
    data_frame = pd.DataFrame()
    data_frame['sentence'] = df['resume']
    data_frame['label'] = df['category']
    sentences = data_frame.sentence.values
    sentences = ["[CLS] " + sentence + " [SEP]" for sentence in
sentences]
    labels = data_frame.label.values
    tokenizer = BertTokenizer.from_pretrained('bert-base-uncased',
do_lower_case=True)
    tokenized_sentences = [tokenizer.tokenize(sentence) for sentence in
sentences]
    input_id_sequences =
[tokenizer.convert_tokens_to_ids(tokenized_sentence) for
tokenized_sentence in tokenized_sentences]
    input_id_sequences = pad_sequences(input_id_sequences,
maxlen=token_length, dtype="long", truncating="post", padding="post")
    attention_masks = [[float(i > 0) for i in input_id_sequence] for
input_id_sequence in input_id_sequences]
    training_dataset, validation_dataset, training_labels, validation_labels,
training_masks, validation_masks = train_test_split(input_id_sequences,
    labels, attention_masks, random_state=2018, test_size=0.1)
    label_encoder = LabelEncoder()
    training_labels = label_encoder.fit_transform(training_labels)
    validation_labels = label_encoder.transform(validation_labels)
    training_labels = np.array(training_labels, dtype=np.int64)
    validation_labels = np.array(validation_labels, dtype=np.int64)
    training_masks = np.array(training_masks, dtype=np.int64)
    validation_masks = np.array(validation_masks, dtype=np.int64)
    training_dataset = np.array(training_dataset, dtype=np.int64)
    validation_dataset = np.array(validation_dataset, dtype=np.int64)
    batch_size = 16
    training_dataset =
TensorDataset(torch.tensor(training_dataset).to(device),
torch.tensor(training_masks).to(device), torch.tensor(training_labels,
dtype=torch.long).to(device))
    training_sampler = RandomSampler(training_dataset)
    training_dataloader = DataLoader(training_dataset,
sampler=training_sampler, batch_size=batch_size)
    validation_dataset =
TensorDataset(torch.tensor(validation_dataset).to(device),
torch.tensor(validation_masks).to(device), torch.tensor(validation_labels,
dtype=torch.long).to(device))
```

```
    validation_sampler = SequentialSampler(validation_dataset)
    validation_dataloader = DataLoader(validation_dataset,
sampler=validation_sampler, batch_size=batch_size)
    num_classes = len(np.unique(training_labels))
    return training_dataloader, validation_dataloader

training_dataloader, validation_dataloader =
load_data("/kaggle/input/nlp-clean/BERT_data_final.csv", 128)
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased",
do_lower_case=True)
encoding = tokenizer(
    df["resume"].tolist(),
    truncation=True,
    padding="max_length",
    max_length=128,
    return_tensors="pt"
)

train_dataloader = DataLoader(training_dataset, batch_size=16,
num_workers=4, pin_memory=True)
val_dataloader = DataLoader(validation_dataset, batch_size=16,
num_workers=4, pin_memory=True)
from torch.optim.lr_scheduler import CosineAnnealingLR

model_opti = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=num_classes,
    hidden_dropout_prob=0.3,  # Increased to 0.3 for better regularization
    attention_probs_dropout_prob=0.3  # Prevent overfitting in attention
layers
)

model_opti = nn.DataParallel(model_opti)
model_opti.to(device)

parameters = list(model_opti.named_parameters())
no_decay = ['bias', 'LayerNorm.weight']
parameters = [
    {'params': [p for n, p in parameters if not any(nd in n for nd in
no_decay)], 'weight_decay': 0.01},
    {'params': [p for n, p in parameters if any(nd in n for nd in no_decay)],
'weight_decay': 0.0}
]

optimizer = AdamW(parameters, lr=2e-5, correct_bias=False)

# Add a scheduler for learning rate decay
lr_scheduler = CosineAnnealingLR(optimizer, T_max=total_epochs,
eta_min=1e-6)

# Freeze Lower BERT Layers
for param in model_opti.module.bert.embeddings.parameters():
    param.requires_grad = False
for layer in model_opti.module.bert.encoder.layer[:4]:  # Freeze first 8
```

Admin No. _____

```
layers
    for param in layer.parameters():
        param.requires_grad = False
from torch.cuda.amp import autocast, GradScaler

# Enable mixed precision training
scaler = GradScaler()
gradient_accumulation_steps = 4

best_val_acc = 0
patience = 7
counter = 0
best_model_path_opti = "/kaggle/working/best_bert_model_opti.pth"

start = time.time()
for epoch in trange(100, desc="Epoch"):
    model_opti.train()
    training_loss = 0
    training_steps = 0
    training_correct = 0
    training_total = 0

    if epoch == 10:  # Unfreeze all layers after 10 epochs
        for param in model_opti.module.bert.parameters():
            param.requires_grad = True
        print("Unfreezing all BERT layers for full fine-tuning")

    for step, batch in enumerate(training_dataloader):
        inputs = batch[0].to(device)
        attention_masks = batch[1].to(device)
        labels = batch[2].to(device)

        optimizer.zero_grad()

        with autocast():
            outputs = model_opti(inputs, attention_mask=attention_masks,
labels=labels)
            loss = outputs.loss / gradient_accumulation_steps

        scaler.scale(loss).backward()

        if (step + 1) % gradient_accumulation_steps == 0:
            scaler.step(optimizer)
            scaler.update()
            optimizer.zero_grad()

        training_loss += loss.item()
        training_steps += 1

        preds = torch.argmax(outputs.logits, dim=1)
        training_correct += (preds == labels).sum().item()
        training_total += labels.size(0)

    # Compute average training loss & accuracy
```

```
    average_training_loss = training_loss / training_steps
    training_accuracy = training_correct / training_total

    print("Epoch {}: Average Training Loss: {:.4f}".format(epoch+1,
average_training_loss))
    print("Epoch {}: Training Accuracy: {:.4f}".format(epoch+1,
training_accuracy))

    # Validation Step
    model_opti.eval()
    validation_accuracy = 0
    validation_steps = 0

    for batch in validation_dataloader:
        inputs = batch[0].to(device)
        attention_masks = batch[1].to(device)
        labels = batch[2].to(device)

        with torch.no_grad():
            outputs = model_opti(inputs, attention_mask=attention_masks,
labels=labels)

        logits = outputs.logits
        temp_validation_accuracy = (torch.argmax(logits, dim=1) ==
labels).sum().item() / labels.size(0)
        validation_accuracy += temp_validation_accuracy
        validation_steps += 1

    # Compute average validation accuracy
    average_validation_accuracy = validation_accuracy / validation_steps
    print("Epoch {}: Validation Accuracy: {:.4f}".format(epoch+1,
average_validation_accuracy))

    # Save best model
    if average_validation_accuracy > best_val_acc:
        best_val_acc = average_validation_accuracy
        torch.save(model_opti.state_dict(), best_model_path_opti)
        print(f"Best model saved at epoch {epoch+1} with validation
accuracy {best_val_acc:.4f}")
        counter = 0  # Reset patience counter
    else:
        counter += 1
        print(f"No improvement, patience count: {counter}/{patience}")

    # Early Stopping
    if counter >= patience:
        print("Early stopping triggered. Training stopped.")
        break

# Track total training duration
opti_duration = time.time() - start

# Apply quantization after training
best_model_path_quantized =
```

```
"/kaggle/working/best_bert_model_quanti.pth"
model_quantized = torch.quantization.quantize_dynamic(
    model_opti, {torch.nn.Linear}, dtype=torch.qint8
)
torch.save(model_quantized.state_dict(), best_model_path_opti)
print("Quantized BERT saved! (75% smaller, faster inference)")
```

Justification

Bert-base-uncased is still used but with max token length 128 only. Mixed-precision training and gradient accumulation is used to reduce training time and prevent vanishing gradients. The optimized model is then further quantized to reduce model size and increase inference speed.
BERT Fine-tuning total training time: 361.61 seconds.
BERT Fine-tuning avg training time per epoch: 9.77 seconds.
BERT Optimization total training time: 223.26 seconds. BERT Optimization avg training time per epoch: 4.75 seconds.
Inference speed:
Fine-tuned BERT took 0.020581 seconds.
Optimized BERT took 0.0074 seconds.

7. Conclusion

This project explored various Natural Language Processing (NLP) techniques to enhance resume classification and ranking. The preprocessing phase tackled data inconsistencies, eliminated duplicates, and standardized text formatting to ensure optimal input for vectorization and modeling. Among multiple vectorization techniques, TF-IDF emerged as the most effective, balancing computational efficiency with classification accuracy.

While deep learning models such as BERT provided high accuracy, they exhibited signs of overfitting due to the dataset's constraints, necessitating fine-tuning strategies like dropout regularization and layer freezing. The resume ranking system demonstrated the ability to match job roles effectively, with insights gained from different similarity metrics. Finally, optimization techniques, including gradient clipping and learning rate scheduling, further refined the training process.

In conclusion, this project underscores the importance of preprocessing, vectorization choices, and model fine-tuning in NLP-based resume classification. The results suggest that traditional techniques like TF-IDF can still outperform more complex models in certain real-world applications, making it the most suitable approach for this dataset. Future work could focus on integrating domain-specific embeddings or leveraging ensemble methods to further improve classification and ranking performance.

8. Appendix

Cite and reference ALL resources that you used in your assignment / report using APA style. Author/Creator. (Year). Source. URL.